

Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS

dfs_bfs.cpp

```
#include <iostream>

#include <vector>

#include <queue>

#include <omp.h>

using namespace std;

class Graph {

    int V;

    vector<vector<int>>> adj;

public:

    Graph(int V) {

        this->V = V;

        adj.resize(V);

    }

    void addEdge(int u, int v) {

        adj[u].push_back(v);

        adj[v].push_back(u); // undirected graph

    }

    //  Parallel BFS

    void parallelBFS(int start) {

        vector<bool> visited(V, false);

        queue<int> q;

        visited[start] = true;

        q.push(start);
```

```

cout << "\nParallel BFS Traversal: ";

while (!q.empty()) {
    int size = q.size();

    #pragma omp parallel for
    for (int i = 0; i < size; i++) {
        int node = -1;

        #pragma omp critical
        {
            if (!q.empty()) {
                node = q.front();
                q.pop();
                cout << node << " ";
            }
        }

        if (node != -1) {
            for (int neighbor : adj[node]) {
                if (!visited[neighbor]) {
                    #pragma omp critical
                    {
                        if (!visited[neighbor]) {
                            visited[neighbor] = true;
                            q.push(neighbor);
                        }
                    }
                }
            }
        }
    }
}

cout << endl;
}

```

```
//  Parallel DFS Utility
```

```
void parallelDFSUtil(int node, vector<bool> &visited) {
```

```
    bool alreadyVisited;
```

```
    #pragma omp critical
```

```
    {
```

```
        alreadyVisited = visited[node];
```

```
        if (!visited[node]) {
```

```
            visited[node] = true;
```

```
            cout << node << " ";
```

```
        }
```

```
    }
```

```
    if (alreadyVisited) return;
```

```
    #pragma omp parallel for
```

```
    for (int i = 0; i < adj[node].size(); i++) {
```

```
        int neighbor = adj[node][i];
```

```
        if (!visited[neighbor]) {
```

```
            #pragma omp task
```

```
            parallelDFSUtil(neighbor, visited);
```

```
        }
```

```
    }
```

```
}
```

```
//  Parallel DFS
```

```
void parallelDFS(int start) {
```

```
    vector<bool> visited(V, false);
```

```
    cout << "\nParallel DFS Traversal: ";
```

```
    #pragma omp parallel
```

```
    {
```

```

        #pragma omp single
        {
            parallelDFSUtil(start, visited);
        }
    }

    cout << endl;
}

};

int main() {
    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    g.parallelBFS(start);
    g.parallelDFS(start);

```

```

    return 0;
}

// 0
// /\
// 1 2
// /\ \
// 3 4 5

// Enter number of vertices: 6
// Enter number of edges: 5
// Enter edges (u v):
// 0 1
// 0 2
// 1 3
// 1 4
// 2 5
// Enter starting vertex: 0

```

Imp

Command to execute :

```

>> g++ bfs_dfs.cpp -fopenmp -o bfs_dfs.exe
>> .\bfs_dfs.exe

```

Explanation :-

This program implements a **Graph** and performs:

1. **Parallel Breadth First Search (BFS)** using OpenMP
2. **Parallel Depth First Search (DFS)** using OpenMP tasks

1. Header Files

```
#include <iostream>
```

```
#include <vector>
```

```
#include <queue>
```

```
#include <omp.h>
```

Purpose

Header	Use
iostream	Input/output (cin, cout)
vector	Dynamic arrays
queue	BFS queue
omp.h	OpenMP parallel programming

2. Graph Class

```
class Graph {
```

```
    int V;
```

```
    vector<vector<int>> adj;
```

Meaning

- $V \rightarrow$ number of vertices
- $adj \rightarrow$ adjacency list

Example graph:

$0 \rightarrow 1,2$

$1 \rightarrow 0,3,4$

$2 \rightarrow 0,5$

Stored as:

$adj[0] = \{1,2\}$

$adj[1] = \{0,3,4\}$

$adj[2] = \{0,5\}$

3. Constructor

```
Graph(int V) {
```

```
    this->V = V;
```

```
    adj.resize(V);
```

```
}
```

What happens

If $V = 6$

```
adj.resize(6);
```

Creates:

```
adj[0]
```

```
adj[1]
```

```
adj[2]
```

```
adj[3]
```

```
adj[4]
```

```
adj[5]
```

Each stores neighbors of that vertex.

4. Adding Edges

```
void addEdge(int u, int v) {  
    adj[u].push_back(v);  
    adj[v].push_back(u);  
}
```

Because graph is **undirected**:

If input:

0 1

Then:

```
adj[0].push_back(1);
```

```
adj[1].push_back(0);
```

Connection works both ways.

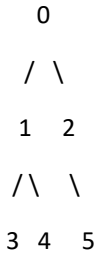
5. Parallel BFS

BFS Concept

BFS explores:

Level by level

Example:



Traversal:

0 → 1 → 2 → 3 → 4 → 5

6. BFS Function

void parallelBFS(int start)

Step 1: Create visited array

vector<bool> visited(V, false);

Initially:

visited = [false,false,false,false,false,false]

Step 2: Create queue

queue<int> q;

Queue stores nodes to process.

Step 3: Mark start visited

visited[start] = true;

q.push(start);

If start = 0:

visited = [true,false,false,false,false,false]

queue = [0]

7. Main BFS Loop

while (!q.empty())

Continue until queue becomes empty.

8. Current Queue Size

```
int size = q.size();
```

Suppose:

```
queue = [1,2]
```

```
size = 2
```

This means current BFS level has 2 nodes.

9. Parallel Processing

```
#pragma omp parallel for
```

```
for (int i = 0; i < size; i++)
```

Multiple threads process nodes simultaneously.

Example:

Thread	Node
--------	------

Thread 1	1
----------	---

Thread 2	2
----------	---

10. Critical Section

```
#pragma omp critical
```

```
{  
    if (!q.empty()) {  
        node = q.front();  
        q.pop();  
        cout << node << " ";  
    }  
}
```

Why critical?

Queue is shared among threads.

Without protection:

- two threads may remove same node
- data corruption may happen

critical allows only ONE thread at a time.

11. Visiting Neighbors

```
for (int neighbor : adj[node])
```

Example:

```
adj[1] = {0,3,4}
```

Loop checks:

- 0
 - 3
 - 4
-

12. Add Unvisited Nodes

```
if (!visited[neighbor])
```

If node not visited:

```
#pragma omp critical
```

```
{
```

```
    if (!visited[neighbor]) {
```

```
        visited[neighbor] = true;
```

```
        q.push(neighbor);
```

```
    }
```

```
}
```

Again critical section prevents:

- duplicate insertion
 - race conditions
-

BFS Execution Example

Initial State

Queue = [0]

Visited = [T,F,F,F,F,F]

Iteration 1

Process node 0

Neighbors:

- 1
- 2

Add them.

Queue = [1,2]

Visited = [T,T,T,F,F,F]

Output = 0

Iteration 2

Parallel threads process:

- 1
- 2

Neighbors added:

- 3
- 4
- 5

Queue = [3,4,5]

Visited = [T,T,T,T,T,T]

Output = 0 1 2

Iteration 3

Process:

- 3
- 4
- 5

Done.

Final BFS:

0 1 2 3 4 5

13. Parallel DFS

DFS Concept

DFS explores deeply before backtracking.

Traversal may be:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 5$

14. DFS Utility Function

```
void parallelDFSUtil(int node, vector<bool> &visited)
```

Recursive DFS helper.

15. Check Visited

```
#pragma omp critical
```

```
{
```

```
    alreadyVisited = visited[node];
```

```
    if (!visited[node]) {
```

```
        visited[node] = true;
```

```
        cout << node << " ";
```

```
    }
```

```
}
```

Critical avoids multiple threads visiting same node simultaneously.

16. Return if Already Visited

```
if (alreadyVisited) return;
```

Stops repeated processing.

17. Parallel Neighbor Processing

```
#pragma omp parallel for
```

```
for (int i = 0; i < adj[node].size(); i++)
```

Different neighbors handled by different threads.

18. OpenMP Tasks

```
#pragma omp task
```

```
parallelDFSUtil(neighbor, visited);
```

Creates independent tasks.

Example:

For node 0:

Task 1 → DFS(1)

Task 2 → DFS(2)

Both may execute simultaneously.

19. Starting DFS

```
#pragma omp parallel
```

```
{
```

```
    #pragma omp single
```

```
    {
```

```
        parallelDFSUtil(start, visited);
```

```
    }
```

```
}
```

Meaning

Directive Purpose

parallel Create thread team

single Only one thread starts recursion

Then tasks spread across threads.

20. Input Example

0 -- 1

| | \

2 3 4

|

5

21. Expected Output

BFS

0 1 2 3 4 5

(Level order)

DFS

Possible output:

0 1 3 4 2 5

OR

0 2 5 1 3 4

Because parallel execution order may vary.

22. Important OpenMP Concepts Used

OpenMP Directive	Meaning
#pragma omp parallel	Create threads
#pragma omp parallel for	Parallel loop
#pragma omp critical	Only one thread enters
#pragma omp single	Single thread executes
#pragma omp task	Create asynchronous task

23. Why Parallelism Helps

For very large graphs:

- multiple nodes explored simultaneously
- faster traversal
- better CPU utilization

Especially useful in:

- social networks
 - web crawling
 - AI search
 - scientific computing
-

24. Limitation of This Program

This implementation is educational.

Real parallel BFS/DFS are more complex because:

- queue locking reduces speed
- many critical sections
- DFS recursion synchronization overhead

Still excellent for learning OpenMP concepts.

DL Practical

Multiclass Classification Using Deep Neural Networks

Example: OCR Letter Recognition Dataset

Dataset: [UCI Letter Recognition Dataset](#)

1. Problem Statement

The goal is to build a **Deep Neural Network (DNN)** that can recognize English alphabet letters (A-Z) using numerical features extracted from images.

This is a **multiclass classification problem** because:

- There are **26 classes**
- Each class represents one letter:

A, B, C, D, ..., Z

The model learns patterns from training data and predicts the correct letter for unseen samples.

2. About the Dataset

The OCR (Optical Character Recognition) Letter Recognition dataset contains:

Property	Value
Total Samples	20,000
Classes	26 letters
Features	16 numerical attributes
Target	Letter (A-Z)

3. Features Description

Each row represents one character image converted into 16 statistical features.

Example features:

Feature Meaning

x-box Horizontal position

y-box Vertical position

width Width of character

height Height of character

onpix Number of pixels

x-bar Mean x position

y-bar Mean y position

4. Workflow of the Project

Dataset → Preprocessing → Encoding → Train/Test Split
→ Build DNN → Train Model → Evaluate Accuracy

Use More Layers

Deeper network can improve accuracy.

Use CNN

Convolutional Neural Networks work even better for OCR/image recognition.

13. Applications of OCR

- Handwriting recognition
 - License plate detection
 - Document digitization
 - Bank cheque reading
 - CAPTCHA solving
-

14. Conclusion

This project demonstrates:

- Multiclass classification using Deep Neural Networks
- OCR letter recognition
- Data preprocessing
- Label encoding
- Neural network training using TensorFlow/Keras

The DNN successfully learns patterns from numerical image features and predicts alphabet letters with high accuracy.

Explanation of the Program

Multiclass Classification Using Deep Neural Network (DNN)

This program uses a **Deep Neural Network (DNN)** to recognize English alphabet letters (A-Z) from the OCR Letter Recognition dataset.

It is a **multiclass classification problem** because the model predicts one class out of **26 possible classes**.

Objective

Train a neural network that:

- Takes 16 numerical features as input
- Learns patterns of letters
- Predicts the correct alphabet (A-Z)

Dataset

Dataset file:

letter-recognition.data

Each row contains:

- First column → letter label (A-Z)

- Remaining 16 columns → numerical features

Example row:

T,2,8,3,5,1,8,13,0,6,6,10,8,0,8,0,8

Where:

- T → output class
- remaining numbers → features

Step-by-Step Explanation

1. Import Libraries

```
import pandas as pd
import numpy as np
```

Purpose

pandas

Used for handling dataset tables.

numpy

Used for numerical operations.

```
from sklearn.model_selection import train_test_split
```

Used to split dataset into:

- training data
- testing data

```
from sklearn.preprocessing import LabelEncoder
```

Used to convert letters into numeric labels.

Example:

Letter Encoded Value

A 0

B 1

C 2

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Used to create Deep Neural Network layers.

2. Load Dataset

```
data = pd.read_csv("letter-recognition.data", header=None)
```

What happens?

Reads the dataset file into a dataframe.

header=None

Dataset has no column names.

3. Split Features and Labels

```
X = data.iloc[:, 1:].values
```

```
y = data.iloc[:, 0].values
```

X

Input features.

```
data.iloc[:, 1:]
```

Means:

- all rows
- columns from index 1 onward

These are the 16 numerical attributes.

y

Target labels.

`data.iloc[:, 0]`

First column contains:

A-Z

4. Encode Labels

```
le = LabelEncoder()
```

```
y = le.fit_transform(y)
```

Neural networks cannot process letters directly.

So:

A → 0

B → 1

C → 2

...

Z → 25

5. Convert Features to Float

```
X = X.astype('float32')
```

Converts feature values into decimal format.

Deep learning models work efficiently with float data.

6. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Purpose

Split data into:

- Training set → model learns
 - Testing set → model evaluation
-

test_size=0.2

20% data used for testing.

80% used for training.

random_state=42

Ensures same random split every time.

7. Build Deep Neural Network

model = Sequential()

Creates a sequential neural network.

Layers are added one after another.

First Hidden Layer

```
model.add(Dense(64, activation='relu', input_shape=(16,)))
```

Explanation

Dense(64)

64 neurons.

activation='relu'

ReLU function:

$f(x) = \max(0, x)$

Adds non-linearity.

input_shape=(16,)

Input layer has:

16 features

Second Hidden Layer

`model.add(Dense(64, activation='relu'))`

Another hidden layer with 64 neurons.

Helps learn deeper patterns.

Output Layer

`model.add(Dense(26, activation='softmax'))`

Why 26?

Because:

26 alphabet classes

Why Softmax?

Softmax converts outputs into probabilities.

Example:

Letter Probability

A 0.01

B 0.02

C 0.90

Prediction:

C

because highest probability.

8. Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Optimizer

```
optimizer='adam'
```

Adam improves learning speed using adaptive gradient descent.

Loss Function

```
loss='sparse_categorical_crossentropy'
```

Used for multiclass classification with integer labels.

Since labels are:

0-25

we use sparse categorical crossentropy.

Metrics

```
metrics=['accuracy']
```

Measures prediction accuracy.

9. Train Model

```
model.fit(X_train, y_train, epochs=25, batch_size=32)
```

Epochs

```
epochs=25
```

Model sees full training data 25 times.

Batch Size

`batch_size=32`

32 samples processed together.

Training Process

During training:

1. Forward propagation
2. Loss calculation
3. Backpropagation
4. Weight update

Repeated for all epochs.

10. Evaluate Model

`loss, acc = model.evaluate(X_test, y_test)`

Tests model on unseen data.

Returns:

- loss
 - accuracy
-

Print Accuracy

`print("Accuracy:", acc)`

Example output:

Accuracy: 0.96

Meaning:

96% correct predictions

Architecture Summary

Input Layer: 16 neurons

Hidden Layer 1: 64 neurons (ReLU)

Hidden Layer 2: 64 neurons (ReLU)

Output Layer: 26 neurons (Softmax)

Flow of Execution

Dataset



Preprocessing



Label Encoding



Train/Test Split



Build DNN



Compile Model



Train Model



Evaluate Accuracy

Why This is Multiclass Classification?

Because:

- there are more than 2 classes
 - output belongs to one of 26 letters
-

Advantages of DNN

- Learns complex patterns
- High accuracy
- Works well for OCR tasks

- Handles multiclass classification efficiently
-

Possible Improvements

1. Feature Scaling

Add:

from sklearn.preprocessing import StandardScaler
to normalize inputs.

2. Add Dropout

Dropout(0.3)

Prevents overfitting.

3. Increase Layers

Deeper networks may improve accuracy.

Time Complexity

Training complexity depends on:

- number of layers
- neurons
- epochs
- dataset size

Approximate complexity:

$O(\text{samples} \times \text{epochs} \times \text{neurons})$

Expected Output

Epoch 1/25

...

Epoch 25/25

...

Accuracy: 0.95

Accuracy may vary slightly each run.

Conclusion

This program demonstrates:

- OCR letter recognition
- Multiclass classification
- Deep Neural Networks using TensorFlow/Keras
- Label encoding
- Training and evaluating a neural network

The DNN learns patterns from numerical character features and predicts alphabet letters with high accuracy.